DEV 140 · DISCUSSION 4 · LAYOUT HOMEWORK

Make Your Page Fit Every Screen

Practice on the sourdough page, then give your own Homework 2 page a Grid or Flexbox layout that changes at phone, tablet, and desktop sizes.

Same `Discussions` folder, same repo, same routine as before. We cover two things: **Grid / Flexbox** (how things sit side by side) and **media queries** (how the layout changes by screen width). On a Mac: use `Cmd` wherever this sheet says `Ctrl`.

YOUR FINISHED LINK

`https://YOUR-USERNAME.github.io/dev-140-yourname-f26/discussion-2/`

your GitHub account | your class repo | your homework folder

```
Desktop\DEV 140 - Web Development\
└── Discussions\                ← open this in VS Code (= your repo)
    ├── discussion-2\           ← your Homework 2 page (the homework)
    │   ├── index.html          ← wrap items in a holder <div> (step 9)
    │   ├── style.css           ← add grid + media queries (steps 10–11)
    │   └── sourdough-loaf.svg
    ├── discussion-3\
    └── discussion-4\           ← NEW: in-class practice (step 2)
        ├── index.html          ← from GitHub (step 4)
        ├── style.css           ← from GitHub (step 4)
        └── sourdough-loaf.svg ← copied (step 3)
```

Before you start ☐ Homework 2 page is live with its `style.css` ☐ VS Code open on `Discussions`, not a subfolder ☐ Read the rubric (page 2)

CSS YOU'LL USE

GRID & FLEXBOX you need at least one

PROPERTY	WHAT IT DOES	EXAMPLE VALUE
GRID: ROWS AND COLUMNS		
`display`	Turns on grid for the holder	`grid`
`grid-template-columns`	One value per column (`1fr` = one equal share)	`1fr 1fr`
FLEXBOX: ONE LINE OF THINGS		
`display`	Lines the items up in a row	`flex`
`flex-wrap`	Lets items drop to a new line if they don't fit	`wrap`
BOTH		
`gap`	Space between the items	`16px`

```
<div class="card-grid">          ← the holder: grid rules go on .card-grid
  <article class="card">…</article>
  <article class="card">…</article>
</div>
```

One line of things? → Flexbox

Nav links, a row of buttons. Each item is as wide as its content.

Line up in columns? → Grid

Cards, a photo gallery. Every item lines up with the ones above it.

MEDIA QUERIES change the layout by screen width

```
@media (min-width: 768px) {      "Is the screen at least 768px wide?"
  .card-grid {                     YES: use the rules inside
    grid-template-columns: 1fr 1fr;
  }
}                                  NO: skip them
```

SCREEN	WHICH CSS	COLUMNS	PRACTICE COLOR
Phone 320px	normal CSS (no media query)	1	cream
Tablet 768px	`@media (min-width: 768px)`	2	light blue
Desktop 1200px	`@media (min-width: 1200px)`	4	light green

- **Phone first.** Write the 1-column layout as normal CSS, then add media queries for bigger screens.
- **Media queries go at the bottom** of `style.css`, after the normal rules, or the normal rules win.
- **Two closing braces** `} }`: one for the rule, one for the media query.
- **The 320px fix:** `img { max-width: 100%; height: auto; }` stops images sticking out the side.
- The background colors are only there so you can *see* which media query is on. For the homework, the **columns** changing is what counts.

GET THE PRACTICE FILES

1. **Open the class folder on GitHub.**
   Click **discussion-4 on GitHub**. You'll see `index.html` and `style.css`.
2. **Make your `discussion-4` folder**
   inside `Discussions`, next to `discussion-3`.
3. **Copy the image.**
   Copy `sourdough-loaf.svg` from your `discussion-3` folder into `discussion-4`.
4. **Paste both files.**
   For each file: open it on GitHub → **Copy raw file**. In VS Code, **New File** in `discussion-4` with the same name, `Ctrl`+`V`, save.

CODE ALONG

5. **Open with Live Server.**
   Right-click `discussion-4/index.html` → **Open with Live Server**.
6. **Turn on the device toolbar.**
   `F12`, then `Ctrl`+`Shift`+`M`. Pick **Responsive** and type a width: 320, 768, or 1200.
7. **Example 1: cards in a grid.**
   Fill in the `.card-grid` TODOs: `display`, columns, `gap`. Then try 320px: squished!
8. **Example 2: media queries.**
   Fix the image, change the columns to `1fr`, then add the 768px and 1200px media queries. Watch the color change at each width.

YOUR HOMEWORK 2 PAGE

9. **Pick a section with 2+ similar items**
   (cards, photos, features) and wrap them in a `<div class="...">` holder.
10. **Add grid or flex to the holder**
    in `discussion-2/style.css`. Start with 1 column for phones.
11. **Add the image fix and media queries**
    at the bottom: more columns at 768px, more again at 1200px.
12. **Commit and sync.**
    `Ctrl`+`Shift`+`G`, type `Make HW2 page responsive`, **Commit**, then **Sync Changes**.
13. **Check and submit.**
    After the green ✓ in **Actions**, test your live link at all three widths. Submit your repo URL.

Test all three widths before you submit: 320, 768, and 1200px.

The page is graded at each one. Any sideways scrolling or overlapping costs points, and 768px must look different from both the phone and the desktop layout.

DEV 140 · DISCUSSION 4 · LAYOUT HOMEWORK

Responsive Layout Rubric

50 points. Your page is graded at three widths: **320px** (phone), **768px** (tablet), and **1200px** (desktop). It needs a Grid or Flexbox section and a media query that visibly changes the layout.

CRITERIA	FULL CREDIT	PARTIAL CREDIT	NO CREDIT
Flexbox or Grid Layout	**15 pts** Flexbox or Grid is applied to a multi-column layout with correct container and item properties. The layout visually demonstrates the tool was used intentionally.	**8 pts** Flexbox or Grid is present but applied incorrectly or only to a single element with no meaningful layout effect.	**0 pts** No Flexbox or Grid used: layout is achieved with floats, positioning, or margin hacks only.
Media Query Present	**10 pts** At least one @media query with a valid breakpoint is present and visibly changes the layout when that breakpoint is crossed.	**5 pts** Media query syntax is present but the breakpoint value is incorrect, or the styles inside it don't visibly change the layout.	**0 pts** No @media query present.
Mobile Viewport (320px)	**10 pts** At 320px, all content is readable, nothing overflows the viewport, and the layout is single-column or appropriately stacked.	**5 pts** Mostly usable at 320px but one element causes overflow or overlapping content.	**0 pts** Layout breaks at 320px: significant overflow, text unreadable, or layout completely broken.
Tablet Viewport (768px)	**8 pts** At 768px, the layout reflects a deliberate intermediate state: not identical to mobile or desktop, or gracefully transitions between them.	**4 pts** Layout at 768px works but looks identical to mobile or desktop: no tablet-specific adaptation.	**0 pts** Layout is broken or unreadable at 768px.
Desktop Viewport (1200px)	**7 pts** At 1200px, the layout uses available space well: multi-column, balanced, nothing stretching excessively wide.	**3 pts** Layout is functional at 1200px but text lines are excessively long, or content is too narrow and doesn't use the available width.	**0 pts** Layout is broken or looks unstyled at 1200px.

WHAT THE GRADER CHECKS, IN PLAIN WORDS

- ☐ **Grid or Flexbox makes columns.** `display: grid` or `display: flex` is on a holder with 2+ items, and they really sit side by side on a big screen. Not just on one lonely element.
- ☐ **Phone (320px): one column, nothing sticks out.** No sideways scrolling, no overlapping, text easy to read. Images have `max-width: 100%`.
- ☐ **Desktop (1200px): uses the space.** Multiple columns, and text doesn't stretch across the whole screen. A `max-width` on the body keeps lines readable.
- ☐ **A media query you can see.** At least one `@media (min-width: ...)` that changes the layout (like the number of columns) when you cross that width. A color change alone isn't a layout change.
- ☐ **Tablet (768px): its own in-between look.** For example 2 columns, when the phone has 1 and the desktop has 3 or 4.
- ☐ **The grader can see it.** `style.css` is pushed, the repo is **Public**, and you submit the repo URL. Test your *live* link, not just Live Server.

IF SOMETHING GOES WRONG

Before you submit: open your live link, press `F12` then `Ctrl`+`Shift`+`M`, and check 320, 768, and 1200. That's exactly what the grader does.

YOU SEE	DO THIS
Grid does nothing	Put `display: grid` on the **holder**, not on each item. Does the class name in the CSS match the HTML?
Media query never switches on	Move it to the **bottom** of `style.css`. If it's above the normal rule, the normal rule wins.
Everything after the media query breaks	Missing the second `}`. Each media query needs two.
Works on a laptop, not on a real phone	Add `<meta name="viewport" content="width=device-width, initial-scale=1.0">` to your `<head>`.
Sideways scrolling or zoomed-out page at 320px	Add `img { max-width: 100%; height: auto; }`. Look for a fixed width like `width: 900px`.
768px looks the same as 320 or 1200	Add a second media query so the tablet gets its own column count.
Can't find the width box	`F12`, `Ctrl`+`Shift`+`M`, then set the top dropdown to **Responsive**.
Works locally, not on GitHub	Commit and sync `style.css`, wait for the green ✓ in **Actions**, then hard refresh with `Ctrl`+`Shift`+`R`.